Journal of Machine Learning Research 9 (2008) 1871-1874 Submitted 5/08; Published 8/08

LIBLINEAR: A Library for Large Linear Classification

Rong-En Fan b90098@csie.ntu.edu.tw
Kai-Wei Chang b92084@csie.ntu.edu.tw
Cho-Jui Hsieh b92085@csie.ntu.edu.tw
Xiang-Rui Wang r95073@csie.ntu.edu.tw
Chih-Jen Lin cjlin@csie.ntu.edu.tw
Department of Computer Science
National Taiwan University
Taipei 106, Taiwan

Editor: Soeren Sonnenburg

Abstract

LIBLINEAR is an open source library for large-scale linear classification. It supports logistic regression and linear support vector machines. We provide easy-to-use command-line tools and library calls for users and developers. Comprehensive documents are available for both beginners and advanced users. Experiments demonstrate that LIBLINEAR is very efficient on large sparse data sets.

Keywords: large-scale linear classification, logistic regression, support vector machines, open source, machine learning

1. Introduction

Solving large-scale classification problems is crucial in many applications such as text classification. Linear classification has become one of the most promising learning techniques for large sparse data with a huge number of instances and features. We develop LIBLINEAR as an easy-to-use tool to deal with such data. It supports L2-regularized logistic regression (LR), L2-loss and L1-loss linear support vector machines (SVMs) (Boser et al., 1992). It inherits many features of the popular SVM library LIBSVM (Chang and Lin, 2001) such as simple usage, rich documentation, and open source license (the BSD license[1]). LIBLINEAR is very efficient for training large-scale problems. For example, it takes only several *seconds* to train a text classification problem from the Reuters Corpus Volume 1 (rcv1) that has more than 600,000 examples. For the same task, a general SVM solver such as LIBSVM would take several hours. Moreover, LIBLINEAR is competitive with or even faster than state of the art linear classifiers such as Pegasos (Shalev-Shwartz et al., 2007) and SVM^{perf} (Joachims, 2006). The software is available at `http://www.csie.ntu.edu.tw/~cjlin/liblinear`.

This article is organized as follows. In Sections 2 and 3, we discuss the design and implementation of LIBLINEAR. We show the performance comparisons in Section 4. Closing remarks are in Section 5.

1. The New BSD license approved by the Open Source Initiative.

©2008 Rong-En Fan, Kai-Wei Chang, Cho-Jui Hsieh, Xiang-Rui Wang and Chih-Jen Lin.

2. Large Linear Classification (Binary and Multi-class)

LIBLINEAR supports two popular binary linear classifiers: LR and linear SVM. Given a set of instance-label pairs $(\boldsymbol{x}_i, y_i), i = 1, \ldots, l,\ \boldsymbol{x}_i \in R^n,\ y_i \in \{-1, +1\}$, both methods solve the following unconstrained optimization problem with different loss functions $\xi(\boldsymbol{w}; \boldsymbol{x}_i, y_i)$:

$$\min_{\boldsymbol{w}} \quad \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum\nolimits_{i=1}^{l} \xi(\boldsymbol{w}; \boldsymbol{x}_i, y_i), \tag{1}$$

where $C > 0$ is a penalty parameter. For SVM, the two common loss functions are $\max(1 - y_i\boldsymbol{w}^T\boldsymbol{x}_i, 0)$ and $\max(1-y_i\boldsymbol{w}^T\boldsymbol{x}_i, 0)^2$. The former is referred to as L1-SVM, while the latter is L2-SVM. For LR, the loss function is $\log(1+e^{-y_i\boldsymbol{w}^T\boldsymbol{x}_i})$, which is derived from a probabilistic model. In some cases, the discriminant function of the classifier includes a bias term, b. LIBLINEAR handles this term by augmenting the vector $\boldsymbol{w}$ and each instance $\boldsymbol{x}_i$ with an additional dimension: $\boldsymbol{w}^T \leftarrow [\boldsymbol{w}^T, b], \boldsymbol{x}_i^T \leftarrow [\boldsymbol{x}_i^T, B]$, where B is a constant specified by the user. The approach for L1-SVM and L2-SVM is a coordinate descent method (Hsieh et al., 2008). For LR and also L2-SVM, LIBLINEAR implements a trust region Newton method (Lin et al., 2008). The Appendix of our SVM guide.[2] discusses when to use which method. In the testing phase, we predict a data point $\boldsymbol{x}$ as positive if $\boldsymbol{w}^T\boldsymbol{x} > 0$, and negative otherwise. For multi-class problems, we implement the one-vs-the-rest strategy and a method by Crammer and Singer. Details are in Keerthi et al. (2008).

3. The Software Package

The LIBLINEAR package includes a library and command-line tools for the learning task. The design is highly inspired by the LIBSVM package. They share similar usage as well as application program interfaces (APIs), so users/developers can easily use both packages. However, their models after training are quite different (in particular, LIBLINEAR stores $\boldsymbol{w}$ in the model, but LIBSVM does not.). Because of such differences, we decide not to combine these two packages together. In this section, we show various aspects of LIBLINEAR.

3.1 Practical Usage

To illustrate the training and testing procedure, we take the data set news20,[3] which has more than one million features. We use the default classifier L2-SVM.

```
$ train news20.binary.tr
[output skipped]
$ predict news20.binary.t news20.binary.tr.model prediction
Accuracy = 96.575% (3863/4000)
```

The whole procedure (training and testing) takes less than 15 seconds on a modern computer. The training time without including disk I/O is less than one second. Beyond this simple way of running LIBLINEAR, several parameters are available for advanced use. For example, one may specify a parameter to obtain probability outputs for logistic regression. Details can be found in the `README` file.

2. The guide can be found at `http://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf`.
3. This is the news20.binary set from `http://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets`. We use a 80/20 split for training and testing.

3.2 Documentation

The LIBLINEAR package comes with plenty of documentation. The `README` file describes the installation process, command-line usage, and the library calls. Users can read the "Quick Start" section, and begin within a few minutes. For developers who use LIBLINEAR in their software, the API document is in the "Library Usage" section. All the interface functions and related data structures are explained in detail. Programs `train.c` and `predict.c` are good examples of using LIBLINEAR APIs. If the `README` file does not give the information users want, they can check the online FAQ page.[4] In addition to software documentation, theoretical properties of the algorithms and comparisons to other methods are in Lin et al. (2008) and Hsieh et al. (2008). The authors are also willing to answer any further questions.

3.3 Design

The main design principle is to keep the whole package as simple as possible while making the source codes easy to read and maintain. Files in LIBLINEAR can be separated into source files, pre-built binaries, documentation, and language bindings. All source codes follow the C/C++ standard, and there is no dependency on external libraries. Therefore, LIBLINEAR can run on almost every platform. We provide a simple `Makefile` to compile the package from source codes. For Windows users, we include pre-built binaries.

Library calls are implemented in the file `linear.cpp`. The `train()` function trains a classifier on the given data and the `predict()` function predicts a given instance. To handle multi-class problems via the one-vs-the-rest strategy, `train()` conducts several binary classifications, each of which is by calling the `train_one()` function. `train_one()` then invokes the solver of users' choice. Implementations follow the algorithm descriptions in Lin et al. (2008) and Hsieh et al. (2008). As LIBLINEAR is written in a modular way, a new solver can be easily plugged in. This makes LIBLINEAR not only a machine learning tool but also an experimental platform.

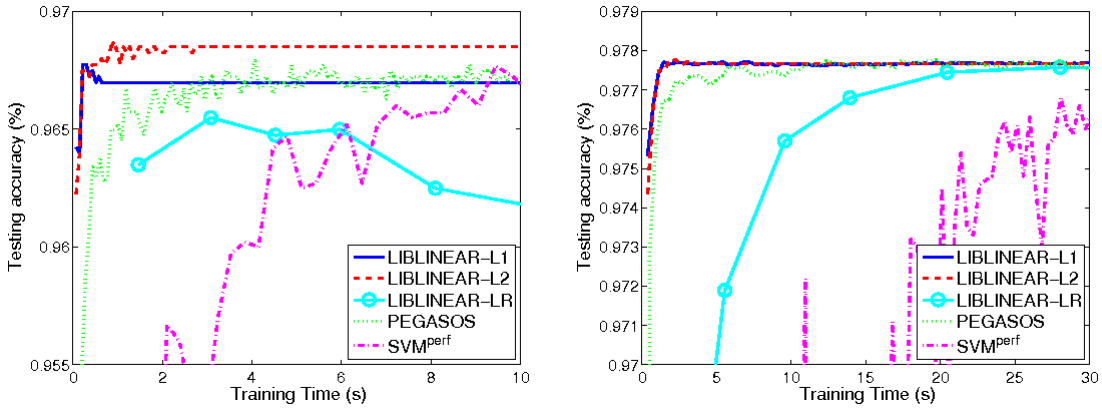
Making extensions of LIBLINEAR to languages other than C/C++ is easy. Following the same setting of the LIBSVM MATLAB/Octave interface, we have a MATLAB/Octave extension available within the package. Many tools designed for LIBSVM can be reused with small modifications. Some examples are the parameter selection tool and the data format checking tool.

4. Comparison

Due to space limitation, we skip here the full details, which are in Lin et al. (2008) and Hsieh et al. (2008). We only demonstrate that LIBLINEAR quickly reaches the testing accuracy corresponding to the optimal solution of (1). We conduct five-fold cross validation to select the best parameter C for each learning method (L1-SVM, L2-SVM, LR); then we train on the whole training set and predict the testing set. Figure 1 shows the comparison between LIBLINEAR and two state of the art L1-SVM solvers: Pegasos (Shalev-Shwartz et al., 2007) and SVM^{perf} (Joachims, 2006). Clearly, LIBLINEAR is efficient.

To make the comparison reproducible, codes used for experiments in Lin et al. (2008) and Hsieh et al. (2008) are available at the LIBLINEAR web page.

4. FAQ can be found at `http://www.csie.ntu.edu.tw/~cjlin/liblinear/FAQ.html`.



(a) news20, l: 19,996, n: 1,355,191, #nz: 9,097,916 (b) rcv1, l: 677,399, n: 47,236, #nz: 156,436,656

Figure 1: Testing accuracy versus training time (in seconds). Data statistics are listed after the data set name. l: number of instances, n: number of features, #nz: number of nonzero feature values. We split each set to 4/5 training and 1/5 testing.

5. Conclusions

LIBLINEAR is a simple and easy-to-use open source package for large linear classification. Experiments and analysis in Lin et al. (2008), Hsieh et al. (2008) and Keerthi et al. (2008) conclude that solvers in LIBLINEAR perform well in practice and have good theoretical properties. LIBLINEAR is still being improved by new research results and suggestions from users. The ultimate goal is to make easy learning with huge data possible.

References

B. E. Boser, I. Guyon, and V. Vapnik. A training algorithm for optimal margin classifiers. In *COLT*, 1992.

C.-C. Chang and C.-J. Lin. *LIBSVM: a library for support vector machines*, 2001. Software available at `http://www.csie.ntu.edu.tw/~cjlin/libsvm`.

Koby Crammer and Yoram Singer. On the learnability and design of output codes for multiclass problems. In *COLT*, 2000.

C.-J. Hsieh, K.-W. Chang, C.-J. Lin, S. S. Keerthi, and S. Sundararajan. A dual coordinate descent method for large-scale linear SVM. In *ICML*, 2008.

T. Joachims. Training linear SVMs in linear time. In *ACM KDD*, 2006.

S. S. Keerthi, S. Sundararajan, K.-W. Chang, C.-J. Hsieh, and C.-J. Lin. A sequential dual method for large scale multi-class linear SVMs. In *ACM KDD*, 2008.

C.-J. Lin, R. C. Weng, and S. S. Keerthi. Trust region Newton method for large-scale logistic regression. *JMLR*, 9:627–650, 2008.

S. Shalev-Shwartz, Y. Singer, and N. Srebro. Pegasos: primal estimated sub-gradient solver for SVM. In *ICML*, 2007.

Appendix: Implementation Details

Appendix A. L1- and L2-SVM

See Hsieh et al. (2008) for details.

Appendix B. Logistic Regression and L2-SVM

See Lin et al. (2008) for details.

Appendix C. Multi-class SVM by Crammer and Singer

Keerthi et al. (2008) extend the coordinate descent method to a sequential dual method for a multi-class SVM formulation by Crammer and Singer. However, our implementation is slightly different from the one in Keerthi et al. (2008). In the following sections, we describe the formulation and the implementation details, including the stopping condition (Appendix C.4) and the shrinking strategy (Appendix C.5).

C.1 Formulations

Given a set of instance-label pairs $(\boldsymbol{x}_i, y_i), i = 1, \ldots, l, \boldsymbol{x}_i \in R^n, y_i \in \{1, \ldots, k\}$, Crammer and Singer (2000) proposed a multi-class approach by solving the following optimization problem:

$$\begin{aligned} \min_{\boldsymbol{w}_m, \xi_i} \quad & \frac{1}{2} \sum_{m=1}^{k} \boldsymbol{w}_m^T \boldsymbol{w}_m + C \sum_{i=1}^{l} \xi_i \\ \text{subject to} \quad & \boldsymbol{w}_{y_i}^T \boldsymbol{x}_i - \boldsymbol{w}_m^T \boldsymbol{x}_i \geq e_i^m - \xi_i, \ i = 1, \ldots, l, \end{aligned} \tag{2}$$

where

$$e_i^m = \begin{cases} 0 & \text{if } y_i = m, \\ 1 & \text{if } y_i \neq m. \end{cases}$$

The decision function is

$$\arg \max_{m=1,\ldots,k} \boldsymbol{w}_m^T \boldsymbol{x}.$$

The dual of (2) is:

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2} \sum_{m=1}^{k} \|\boldsymbol{w}_m\|^2 + \sum_{i=1}^{l} \sum_{m=1}^{k} e_i^m \alpha_i^m \\ \text{subject to} \quad & \sum_{m=1}^{k} \alpha_i^m = 0, \forall i = 1, \ldots, l \\ & \alpha_i^m \leq C_{y_i}^m, \forall i = 1, \ldots, l, m = 1, \ldots, k, \end{aligned} \tag{3}$$

where

$$\boldsymbol{w}_m = \sum_{i=1}^{l} \alpha_i^m \boldsymbol{x}_i, \forall m, \quad \bar{\boldsymbol{\alpha}}_i = [\alpha_i^1, \ldots, \alpha_i^k]^T, \quad \boldsymbol{\alpha} = [\alpha_1^1, \ldots, \alpha_1^k, \ldots, \alpha_l^1, \ldots, \alpha_l^k]^T. \tag{4}$$

and

$$C_{y_i}^m = \begin{cases} 0 & \text{if } y_i \neq m, \\ C & \text{if } y_i = m. \end{cases} \tag{5}$$

Recently, Keerthi et al. (2008) proposed a sequential dual method to efficiently solve (3). Our implementation is based on this paper. The main differences are the sub-problem solver and the shrinking strategy.

C.2 The Sequential Dual Method for (3)

The optimization problem (3) has kl variables, which are very large. Therefore, we extend the coordinate descent method to decomposes $\boldsymbol{\alpha}$ into blocks $[\bar{\boldsymbol{\alpha}}_1, \ldots, \bar{\boldsymbol{\alpha}}_l]$. Each time we select an index i and aim at minimizing the following sub-problem formed by $\bar{\boldsymbol{\alpha}}_i$:

$$\begin{aligned} \min_{\bar{\boldsymbol{\alpha}}_i} \quad & \sum_{m=1}^{k} \frac{1}{2} A(\alpha_i^m)^2 + B_m \alpha_i^m \\ \text{subject to} \quad & \sum_{m=1}^{k} \alpha_i^m = 0, \\ & \alpha_i^m \leq C_{y_i}^m, m = \{1, \ldots, k\}, \end{aligned}$$

where

$$A = \boldsymbol{x}_i^T \boldsymbol{x}_i \text{ and } B_m = \boldsymbol{w}_m^T \boldsymbol{x}_i + e_i^m - A\alpha_i^m.$$

Since bounded variables (i.e., $\alpha_i^m = C_{y_i}^m, \forall m \notin U_i$) can be shrunken during training, we minimize with respect to a sub-vector $\bar{\boldsymbol{\alpha}}_i^{U_i}$, where $U_i \subset \{1, \ldots, k\}$ is an index set. That is, we solve the following $|U_i|$-variable sub-problem while fixing other variables:

$$\begin{aligned} \min_{\bar{\boldsymbol{\alpha}}_i^{U_i}} \quad & \sum_{m \in U_i} \frac{1}{2} A(\alpha_i^m)^2 + B_m \alpha_i^m \\ \text{subject to} \quad & \sum_{m \in U_i} \alpha_i^m = - \sum_{m \notin U_i} \alpha_i^m, \\ & \alpha_i^m \leq C_{y_i}^m, m \in U_i. \end{aligned} \tag{6}$$

Notice that there are two situations that we do not solve the sub-problem of index i. First, if $|U_i| < 2$, then the whole $\bar{\boldsymbol{\alpha}}_i$ is fixed by the equality constraint in (6). So we can shrink the whole vector $\bar{\boldsymbol{\alpha}}_i$ while training. Second, if $A = 0$, then $\boldsymbol{x}_i = \mathbf{0}$ and (4) shows that the value of α_i^m does not affect $\boldsymbol{w}_m$ for all m. So the value of $\bar{\boldsymbol{\alpha}}_i$ is independent of other variables and does not affect the final model. Thus we do not need to solve $\bar{\boldsymbol{\alpha}}_i$ for those $\boldsymbol{x}_i = \mathbf{0}$.

Similar to Hsieh et al. (2008), we consider a random permutation heuristic. That is, instead of solving sub-problems in the order of $\bar{\boldsymbol{\alpha}}_1, \ldots \bar{\boldsymbol{\alpha}}_l$, we permute $\{1, \ldots l\}$ to $\{\pi(1), \ldots \pi(l)\}$, and solve sub-problems in the order of $\bar{\boldsymbol{\alpha}}_{\pi(1)}, \ldots, \bar{\boldsymbol{\alpha}}_{\pi(l)}$. Past results show that such a heuristic gives faster convergence.

We discuss our sub-problem solver in Appendix C.3. After solving the sub-problem, if $\hat{\alpha}_i^m$ is the old value and α_i^m is the value after updating, we maintain $\boldsymbol{w}_m$, defined in (4), by

$$\boldsymbol{w}_m \leftarrow \boldsymbol{w}_m + (\alpha_i^m - \hat{\alpha}_i^m) y_i \boldsymbol{x}_i. \tag{7}$$

Algorithm 1 The coordinate descent method for (3)

- Given $\boldsymbol{\alpha}$ and the corresponding $\boldsymbol{w}_m$
- While $\boldsymbol{\alpha}$ is not optimal, (outer iteration)
  1. Randomly permute $\{1, \ldots, l\}$ to $\{\pi(1), \ldots, \pi(l)\}$
  2. For $i = \pi(1), \ldots, \pi(l)$, (inner iteration)

     If $\bar{\boldsymbol{\alpha}}_i$ is active and $\boldsymbol{x}_i^T \boldsymbol{x}_i \neq 0$ (i.e., $A \neq 0$)
     - Solve a $|U_i|$-variable sub-problem (6)
     - Maintain $\boldsymbol{w}_m$ for all m by (7)

To save the computational time, we collect elements satisfying $\alpha_i^m \neq \hat{\alpha}_i^m$ before doing (7). The procedure is described in Algorithm 1.

C.3 Solving the sub-problem (6)

We follow the approach in Crammer and Singer (2000). Let $\boldsymbol{\nu} = A\bar{\boldsymbol{\alpha}}_i^{U_i} + \boldsymbol{B}$. Solving (6) is equivalent to solving

$$\begin{aligned} \min_{\boldsymbol{\nu}} \quad & \frac{1}{2}\|\boldsymbol{\nu}\|^2 \\ \text{subject to} \quad & \nu_m \leq AC_{y_i}^m + B_m, \forall m \in U_i, \\ & \sum_{m \in U_i} \nu_m = \sum_{m \in U_i} B_m - A \sum_{m \notin U_i} C_{y_i}^m. \end{aligned} \tag{8}$$

By defining $\boldsymbol{D}$ as

$$D_m = \begin{cases} B_m + AC_{y_i}^m & \text{if } \ m = y_i, \\ B_m & \text{if } \ m \neq y_i, \end{cases} \tag{9}$$

Eq. (8) becomes

$$\begin{aligned} \min_{\boldsymbol{\nu}} \quad & \frac{1}{2}\|\boldsymbol{\nu}\|^2 \\ \text{subject to} \quad & \nu_m \leq D_m, \ \forall m \in U_i, \\ & \sum_{m \in U_i} \nu_m = \sum_{m \in U_i} D_m - \sum_{m \in U_i} AC_{y_i}^m - A \sum_{m \notin U_i} C_{y_i}^m = \sum_{m \in U_i} D_m - AC. \end{aligned} \tag{10}$$

Note that the last equality is from (5). The KKT optimality conditions of (10) are

$$\begin{aligned} \nu_m &= \beta - \rho_m, \\ \rho_m(D_m - \nu_m) &= 0, \rho_m \geq 0, \forall m \in U_i, \\ \sum_{m \in U_i} \nu_m &= (\sum_{m \in U_i} D_m) - AC, \end{aligned} \tag{11}$$

Algorithm 2 Solving the sub-problem

- Given A, $\boldsymbol{B}$
- Compute $\boldsymbol{D}$ by (9)
- Sort $\boldsymbol{D}$ in decreasing order; assume $\boldsymbol{D}$ has elements $D_0, D_1, \ldots, D_{|U_i|-1}$
- $r \leftarrow 1, \beta \leftarrow D_0 - AC$
- While $r < |U_i|$ and $\beta/r < D_r$
  1. $\beta \leftarrow \beta + D_r$
  2. $r \leftarrow r + 1$
- $\alpha_i^m \leftarrow \min(C_{y_i}^m, (\beta - B_m)/A), \forall m$

where β and ρ_m are Lagrange multipliers. Eq. (11) implies

$$\sum_{\rho_m=0} \beta + \sum_{\rho_m \neq 0} D_m = (\sum_{m \in U_i} D_m) - AC, \text{and } \beta \geq D_m, \forall \rho_m \neq 0.$$

Thus we intend to find a β, where

$$\beta = \frac{(\sum_{\rho_m=0} D_m) - AC}{|\{m \mid \rho_m = 0\}|}, \text{and } \beta \geq \max_{\rho_m \neq 0} D_m. \tag{12}$$

To solve (12), we need to verify the set $\{m \mid \rho_m = 0\}$, and from (11) and (10), we have:

$$\beta > D_m \text{ if } \rho_m > 0,$$
$$\beta \leq D_m \text{ if } \rho_m = 0.$$

We begin with a set $\Phi = \phi$, and then sequentially add an index m to Φ by the decreasing order of D_m until

$$\frac{(\sum_{m \in \Phi} D_m) - AC}{|\Phi|} \geq \max_{m \notin \Phi} D_m.$$

Having the set $\Phi = \{m \mid \rho_m = 0\}$ and $\beta = \frac{\sum_{m \in \Phi} D_m - AC}{|\Phi|}$, the optimal solution can be computed by the following equation:

$$\alpha_i^m = \min(C_{y_i}^m, (\beta - B_m)/A).$$

Algorithm 2 lists the details for solving the sub-problem (6).

C.4 Stopping Condition

The KKT optimality conditions of (3) imply that there are $b_1, \ldots, b_l \in R$ such that for all $i = 1, \ldots, l$, $m = 1, \ldots, k$,

$$\boldsymbol{w}_m^T \boldsymbol{x}_i + e_i^m - b_i = 0 \text{ if } \alpha_i^m < C_i^m,$$
$$\boldsymbol{w}_m^T \boldsymbol{x}_i + e_i^m - b_i \leq 0 \text{ if } \alpha_i^m = C_i^m.$$

Let

$$G_i^m = \frac{\partial f(\boldsymbol{\alpha})}{\partial \alpha_i^m} = \boldsymbol{w}_m^T \boldsymbol{x}_i + e_i^m, \forall i, m,$$

the optimality of $\boldsymbol{\alpha}$ holds if and only if

$$\max_m G_i^m - \min_{m:\alpha_i^m < C_i^m} G_i^m = 0, \forall i. \tag{13}$$

At each inner iteration, we first compute G_i^m and define:

$$\text{minG} \equiv \min_{m:\alpha_i^m < C_i^m} G_i^m, \text{maxG} \equiv \max_m G_i^m, S_i = \text{maxG} - \text{minG}.$$

Then the stopping condition for a tolerance $\epsilon > 0$ can be checked by

$$\max_i S_i < \epsilon. \tag{14}$$

Note that maxG and minG are calculated based on the latest $\boldsymbol{\alpha}$ (i.e., $\boldsymbol{\alpha}$ after each inner iteration).

C.5 Shrinking Strategy

The shrinking technique reduces the size of the problem without considering some bounded variables. Eq. (13) suggests that we can shrink α_i^m out if α_i^m satisfies the following condition:

$$\alpha_i^m = C_{y_i}^m \text{ and } G_i^m < \text{minG}. \tag{15}$$

Then we solve a $|U_i|$-variable sub-problem (6). To implement this strategy, we maintain an $l \times k$ index array `alpha_index` and an l array `activesize_i`, such that `activesize_i[i]` $= |U_i|$. We let the first `activesize_i[i]` elements in `alpha_index[i]` are active indexes, and others are shrunken indexes. Moreover, we need to maintain an l-variable array `y_index`, such that

$$\texttt{alphaindex[i][y_index[i]]} = y_i. \tag{16}$$

When we shrink a index `alpha_index[i][m]` out, we first find the largest $\bar{m}$ such that $\bar{m} <$ `activesize_i[i]` and `alpha_index[i]`$[\bar{\texttt{m}}]$ does not satisfy the shrinking condition (15), then swap the two elements and decrease `activesize_i[i]` by 1. Note that if we swap index y_i, we need to maintain `y_index[i]` to ensure (16). For the instance level shrinking and random permutation, we also maintain a index array `index` and a variable `activesize` similar to `alpha_index` and `activesize_i`, respectively. We let the first `activesize` elements of `index` be active indexes, while others be shrunken indexes. When $|U_i|$, the active size of $\bar{\boldsymbol{\alpha}}_i$, is less than 2 (`activesize_i[i]` < 2), we swap this index with the last active element in `index`, and decrease `activesize` by 1.

However, experiments show that (15) is too aggressive. There are too many wrongly shrunken variables. To deal with this problem, we use an ϵ-cooling strategy. Given a pre-specified stopping tolerance ϵ, we begin with

$$\epsilon_{\text{shrink}} = \max(1, 10\epsilon)$$

and decrease it by a factor of 2 in each graded step until $\epsilon_{\text{shrink}} \leq \epsilon$.

Algorithm 3 Shrinking strategy

- Given ϵ
- Begin with $\epsilon_{\text{shrink}} \leftarrow \max(1, 10\epsilon)$, `start_from_all` $\leftarrow True$
- While
  1. For all active $\bar{\boldsymbol{\alpha}}_i$
     (a) Do shrinking and calculate S_i
     (b) `stopping` $\leftarrow \max(\texttt{stopping}, S_i)$
     (c) Optimize over active variables in $\bar{\boldsymbol{\alpha}}_i$
  2. If `stopping` $< \epsilon_{\text{shrink}}$
     (a) If `stopping` $< \epsilon$ and `start_from_all` is $True$, BREAK
     (b) Take all shrunken variables back
     (c) `start_from_all` $\leftarrow True$
     (d) $\epsilon_{\text{shrink}} \leftarrow \max(\epsilon, \epsilon_{\text{shrink}}/2)$

     Else

     (a) `start_from_all` $\leftarrow False$

The program ends if the stopping condition (14) is satisfied. But we can exactly compute (14) only when there are no shrunken variables. Thus the process stops under the following two conditions:

1. None of the instances is shrunken in the beginning of the loop.
2. (14) is satisfied.

Our shrinking strategy is in Algorithm 3.

Regarding the random permutation, we permute the first `activesize` elements of `index` at each outer iteration, and then sequentially solve the sub-problems.